

Università di Pisa

Dipartimento di Informatica

Generative and Deep Learning

14 simulazioni d'orale

ricostruite su esami reali di giugno e luglio 2026

Le risposte sono in fondo, a partire da pagina 11.

Non arrivarci prima di aver provato a voce.

August 25, 2026

Contents

Istruzioni	3
Come si esegue una simulazione	3
Sulle formule	4
1 Le simulazioni	5
1.1 Self-attention — l’eliminatória	5
1.2 HMM + VAE	5
1.3 Bayesian Networks + esercizio + VAE	6
1.4 CNN + HMM + Bellman — la lode	6
1.5 LSTM + Normalizing Flow — la lode	6
1.6 LDA + GNN — la lode	7
1.7 Boltzmann + RBM + GAN	7
1.8 RNN + ELBO	8
1.9 MRF + Normalizing Flow	8
1.10 MDP + Q-learning + LDA	8
1.11 CNN + RBM	9
1.12 LSTM + Diffusion	9
1.13 GMM + CNN	9
1.14 Grafi + Diffusion — la coppia GrIDDD	10
2 Risposte	11
1 — Self-attention	11
2 — HMM + VAE	11
3 — BN + esercizio + VAE	12
4 — CNN + HMM + Bellman	12
5 — LSTM + Normalizing Flow	12
6 — LDA + GNN	13
7 — Boltzmann + GAN	14
8 — RNN + ELBO	14
9 — MRF + Normalizing Flow	15
10 — MDP + Q-learning + LDA	15
11 — CNN + RBM	15
12 — LSTM + Diffusion	16
13 — GMM + CNN	16
14 — Grafi + Diffusion	16

Istruzioni

Ognuna delle quattordici simulazioni riproduce un orale **realmente sostenuto** negli appelli di giugno e luglio 2026: stessa coppia di argomenti, stesso ordine, e — dove è noto — stesso voto assegnato. Non sono inventate.

Le tre regole, che vengono dai resoconti reali

- 1. La prima domanda è eliminatoria.** “I primi sono usciti tutti per la policy: o lo sai alla prima o esci.” Se su una simulazione non riesci a partire sulla prima domanda, quella simulazione è un esame fallito — non un esercizio in cui recuperare dopo.
- 2. Il voto si fa scendendo, non allargandosi.** Ogni domanda ha tre livelli. Il livello 1 ti tiene dentro, il 2 ti dà il voto medio, il 3 è quello da 30 e lode. Gli argomenti da lode osservati: Bellman optimality, reinforcement learning, e i temi delle ultime lezioni.
- 3. Se le domande si semplificano, il voto sta scendendo.** “Se vedi che comincia a fare domande facili facili, tipo roba sulle CNN, allora il voto sta scendendo: di solito fa così.” Non è gentilezza, sta cercando il tuo livello.

Come si esegue una simulazione

1. Metti un timer. Il budget indicato per ogni simulazione è realistico: circa 20–25 minuti in tutto.
2. **Parla ad alta voce**, con carta e penna accanto. Non pensare le risposte: dille. La differenza fra le due cose è tutto l'esame.
3. Fermati al primo livello su cui non sai partire, e *annota dove*. Non guardare la risposta e non passare alla domanda successiva: la simulazione finisce lì, come finirebbe l'orale.
4. Solo dopo vai alle risposte, e confronta. Poi **richiudi e ridilla**: se la seconda volta non esce fluida, non era acquisita.

Come assegnarti il voto

Bocciato	non parti sul livello 1 della prima domanda.
18–22	arrivi al livello 1 di entrambe, ma vai in difficoltà sul 2.
24–27	livello 2 di entrambe, con imprecisioni sulle formule che sai correggere.
28–30	livello 2 solido e almeno un livello 3 affrontato.
30L	livello 3 su entrambe, formule scritte e <i>spiegate</i> .

Sii severo: chi ha preso 18 agli orali reali si era preparato pensando di essere da 27.

Sulle formule

Il consiglio più utile emerso dai resoconti, e va contro l'istinto:

“Se provi a impararti a memoria tutte le formule ti ammazzi, e lui capisce che hai imparato a memoria. Se sai una formula a memoria lui lo vede e ti ci becca; se sai i passaggi ma sbagli a mettere un indice, magari ti aiuta.”

Il metodo alternativo, descritto per HMM ma generalizzabile: parti dal *disegno* del modello, leggi la likelihood dal disegno, introduci le variabili indicatrici che trasformano le sommatorie in produttorie, prendi il logaritmo, deriva. Non si ricorda il risultato: si ricorda la strada.

Ultimo avvertimento pratico, sempre dai resoconti: *“a volte le formule nelle slide hanno typo o errori: conviene sempre provarsele a derivare da soli”*.

Chapter 1

Le simulazioni

1.1 Self-attention — l'eliminatoria

Simulazione 1

luglio 2026 — esito: BOCCIATO

Argomento unico. L'orale si è chiuso alla prima domanda. Due candidati diversi, in due appelli diversi, sono usciti esattamente su questa. Budget: 10 minuti.

L1. Parlami della self-attention. *Che cos'è un token?*

L2. Come si calcolano Q , K e V ? Scrivi la formula completa dell'attenzione.

L3. Perché si divide per $\sqrt{d_k}$? Che cosa succede senza?

Perché questa simulazione va per prima

Non è difficile: è *elementare*. Ed è quella su cui si è bocciato di più. Se non la superi in scioltezza, tutto il resto del tuo studio è a rischio, perché significa che sai riconoscere e non produrre.

1.2 HMM + VAE

Simulazione 2

luglio 2026 — esito: RITIRATO

Il candidato è andato bene su HMM ma ha fatto un'imprecisione sulla formula del learning che non ha saputo sistemare; su VAE non è riuscito a esporre chiaramente e si è ritirato. Budget: 25 minuti.

D1.L1 Che cos'è un HMM? Disegnalo ed elenca i suoi parametri.

D1.L2 Com'è fatta la congiunta? Scrivila sia in forma generale sia parametrizzata con variabili indicatrici e matrice di transizione.

D1.L3 Deriva le equazioni di Baum-Welch. Come si trovano γ_t e $\xi_{t,t+1}$? Perché servono i moltiplicatori di Lagrange nell'M-step?

D2.L1 Che cos'è un VAE? Qual è la sua loss?

D2.L2 Deriva l'ELBO e mostra come compaiono i due termini. Scrivi la KL in forma chiusa.

D2.L3 Che cos'è il reparametrization trick, quale problema risolve, e che cosa succederebbe senza?

1.3 Bayesian Networks + esercizio + VAE

Simulazione 3

luglio 2026 — esito: 28

Notevole perché contiene un *esercizio* vero, non solo teoria. Budget: 25 minuti.

D1.L1 Definisci una rete Bayesiana. Scrivi la fattorizzazione della congiunta.

D1.L2 Esercizio: dato il DAG $A \rightarrow B \leftarrow C \rightarrow D$, e poi $B \rightarrow E$. Rispetto a quali insiemi di nodi A e D sono condizionalmente indipendenti? Motiva ogni caso.

D1.L3 In una rete Bayesiana abbiamo solo proprietà *locali*? Enuncia la local Markov property e spiega come si collega alla d-separazione.

D2.L1 Che cos'è un VAE e a che cosa serve il termine KL?

D2.L2 Perché un autoencoder classico non è un buon modello generativo, anche con il bottleneck?

D2.L3 Che cos'è il posterior collapse? Quando si verifica e come si mitiga?

1.4 CNN + HMM + Bellman — la lode

Simulazione 4

giugno 2026 — esito: 30 e LODE

Tre argomenti, con la terza domanda usata esplicitamente per la lode. Budget: 30 minuti.

D1.L1 Che cos'è una convoluzione? Come si calcolano le dimensioni dell'output?

D1.L2 Che cos'è il parameter sharing e quale bias induttivo introduce? Equivarianza o invarianza?

D2.L1 Che cos'è un HMM?

D2.L2 Scrivi il termine α dell'algoritmo forward, con inizializzazione, ricorsione e terminazione. Qual è la complessità e perché?

D3.L3 Scrivi la Bellman optimality equation. Perché non è lineare, a differenza della Bellman expectation? Perché l'operatore è una contrazione, e che cosa se ne deduce?

Nota

Il termine α e la Bellman optimality sono le due formule esplicitamente citate nei resoconti come “chieste a giugno”, la seconda *per la lode*. Se punti al 30, queste due vanno sapute alla perfezione.

1.5 LSTM + Normalizing Flow — la lode

Simulazione 5

luglio 2026 — esito: 30 e LODE

Riportata due volte in due appelli diversi, una volta come “LSTM: grafico e formule; Normalizing Flow: tutto tranne il continuous”. Budget: 25 minuti.

D1.L1 Disegna lo schema di una cella LSTM ed elenca le porte.

D1.L2 Scrivi tutte le equazioni. Perché l'aggiornamento della cella è additivo e non moltiplicativo?

D1.L3 Deriva $\partial c_t / \partial c_{t-1}$ e confrontala con la RNN vanilla. Che cosa succede se tutte le forget gate imparano $f_t \approx 0$?

D2.L1 Qual è l'idea di base di un normalizing flow? Su che cosa si fonda?

D2.L2 Perché lo Jacobiano va considerato? Scrivi la formula del cambio di variabile e spiega il segno.

D2.L3 Quali metodi si usano per costruire un flow? Spiega il coupling layer e perché il suo Jacobiano è triangolare. Che cos'è un residual flow e come si stima il suo log-determinante?

1.6 LDA + GNN — la lode

Simulazione 6

luglio 2026 — esito: 30 e LODE

Il caso in cui la scala di scavo è più visibile: quattro livelli dentro LDA. Budget: 25 minuti.

D1.L1 Descrivi LDA: schema e processo generativo.

D1.L2 Come si fa learning in LDA? Che cos'è il posterior approssimato?

D1.L3 Perché in LDA si usa il Gibbs sampling, e come? Scrivi l'equazione di aggiornamento.

D2.L1 Scrivi l'equazione di aggiornamento del learning sui grafi.

D2.L2 A che cosa serve la funzione di aggregazione? Quali requisiti deve soddisfare e che cosa succede se non li soddisfa?

D2.L3 Confronta somma, media e max come aggregatori. Quale raggiunge il limite 1-WL e perché gli altri no?

Attenzione

Un messaggio del 25 luglio riferisce che il professore avrebbe detto di *non* chiedere il training di LDA. Ma questo orale, di pochi giorni dopo, lo contiene. Preparalo comunque: il costo è basso, quello di non saperlo se viene chiesto è l'esame.

1.7 Boltzmann + RBM + GAN

Simulazione 7

luglio 2026 — esito: 26

La catena di scavo più documentata di tutte: il candidato ha riportato esattamente su che cosa è caduto. Budget: 25 minuti.

D1.L1 Che cos'è una Boltzmann machine? E una *restricted* Boltzmann machine?

D1.L2 Scrivi il gradiente della log-verosimiglianza. **Che cos'è l'operatore $\langle \cdot \rangle$?** Perché la fase negativa è il pezzo difficile?

D1.L3 Come si calcola $P(h_j=1 \mid \mathbf{v})$? E $P(v_i=1 \mid \mathbf{h})$? Perché la struttura bipartita permette il block Gibbs?

D2.L1 Scrivi l'obiettivo minimax di una GAN e ricava il discriminatore ottimale.

D2.L2 **Scrivi il sup** nel funzionale del discriminatore. Qual è la differenza fra max e sup?

D2.L3 Allora *perché* nella Wasserstein si usa il sup? E perché D deve essere Lipschitz? (Non “perché è contrattiva”: perché *deve* esserlo.)

1.8 RNN + ELBO

Simulazione 8

luglio 2026 — esito: 28–30

Coppia riportata due volte. Budget: 25 minuti.

D1.L1 Che cos'è una RNN? Qual è il vantaggio del *weight sharing*?

D1.L2 Qual è l'assunzione che rende il weight sharing utilizzabile? Scrivi $\partial h_t / \partial h_{t-\tau}$.

D1.L3 Perché tanh e sigmoide mi fanno perdere informazione? Che cosa mi dice il raggio spettrale, e attraverso quale oggetto?

D2.L1 Che cos'è l'ELBO? Derivalo da $\log p(\mathbf{x})$ con Jensen.

D2.L2 Scrivi l'identità esatta con la KL. Che cosa dice sui tre oggetti che collega?

D2.L3 Come collega EM, inferenza variazionale, VAE e diffusion? Che cosa cambia in ciascuno?

1.9 MRF + Normalizing Flow

Simulazione 9

luglio 2026 — esito: 20

Voto basso: utile per capire dove si perde. Budget: 25 minuti.

D1.L1 Che cos'è un Markov Random Field? Come fattorizza?

D1.L2 Enuncia Hammersley-Clifford. Perché serve l'ipotesi di positività stretta?

D1.L3 Come si relazionano MRF e reti Bayesiane? Fai un esempio di indipendenza rappresentabile da uno e non dall'altro.

D2.L1 Idea di base del normalizing flow.

D2.L2 Scrivi la log-likelihood di un flow a K layer. Quali requisiti sui layer?

D2.L3 Likelihood esatta: e allora perché i flow non hanno vinto?

1.10 MDP + Q-learning + LDA

Simulazione 10

luglio 2026 — candidato con progetto

L'RL è riportato come argomento "da almeno 30". Budget: 30 minuti.

D1.L1 Definisci un MDP. Che cosa significa "Markov" qui?

D1.L2 Che cosa fa γ ? Dai le due giustificazioni. Deriva la Bellman expectation da $G_t = r_{t+1} + \gamma G_{t+1}$.

D1.L3 Perché l'operatore di Bellman è una contrazione? Che cosa se ne deduce su value iteration?

D2.L1 Scrivi l'aggiornamento di Q-learning. Perché è off-policy?

D2.L2 Differenza con SARSA. Quando preferiresti l'uno o l'altro?

D3.L1 Descrivi LDA e il suo processo generativo.

1.11 CNN + RBM

Simulazione 11

luglio 2026 — esito: 25

Budget: 25 minuti.

D1.L1 Convoluzione: definizione, dimensioni dell'output, differenza con la cross-correlazione.

D1.L2 Che cos'è il degradation problem e come lo risolve ResNet? Perché le skip connection aiutano il gradiente?

D1.L3 Come si calcola il campo recettivo dopo 5 layer? Che cosa cambia inserendo un pooling?

D2.L1 Che cos'è una RBM? Scrivi l'energia.

D2.L2 Spiega Contrastive Divergence. Perché è un'approssimazione del gradiente?

D2.L3 CD-1 è il gradiente di qualche funzione obiettivo? Che cosa cambia Persistent CD?

1.12 LSTM + Diffusion

Simulazione 12

luglio 2026 — esito: 27

Budget: 25 minuti.

D1.L1 Schema della cella LSTM e ruolo delle porte.

D1.L2 Differenze principali fra LSTM e GRU. Quale sceglieresti a risorse limitate?

D2.L1 Scrivi il forward process di DDPM. Deriva la forma chiusa $q(\mathbf{z}_t | \mathbf{x}_0)$.

D2.L2 **Scrivi l'ELBO** della catena di diffusione. Quali sono i tre tipi di termini?

D2.L3 Come si arriva alla loss simplified? Che cosa si trascura, ed è neutro?

Attenzione

“Ha scritto male l'ELBO” è il motivo per cui un candidato di fine luglio si è fermato a 25. Questa è la formula che devi saper scrivere per intero, con i tre termini e i loro nomi.

1.13 GMM + CNN

Simulazione 13

luglio 2026 — esito: 18

“Sono andato nel pallone.” Argomenti fra i più semplici del programma, voto minimo: la prova che il problema non è la difficoltà dell'argomento. Budget: 20 minuti.

D1.L1 Che cos'è un GMM? Scrivi la densità.

D1.L2 Deriva E-step e M-step. Che cosa sono le responsabilità e come si interpretano?

D1.L3 Perché EM garantisce l'aumento monotono della log-verosimiglianza? Qual è il passaggio che rende valido l'argomento?

D2.L1 Operazione di convoluzione e conto delle dimensioni.

D2.L2 Quanti parametri ha un layer convoluzionale rispetto a una fully connected equivalente? Fai il conto.

1.14 Grafi + Diffusion — la coppia GrIDDD

Simulazione 14

giugno 2026 — “convolution on graphs” e “diffusion models”

Questa è la simulazione che ti riguarda più delle altre: se vale ancora la regola per cui una domanda cade nell'area dell'ultimo midterm, il tuo poster su GrIDDD (graph diffusion) espone esattamente questi due. Budget: 25 minuti.

D1.L1 Che cos'è la convoluzione su grafi? Perché non si può applicare una CNN a un grafo?

D1.L2 Scrivi la regola di aggiornamento della GCN, in forma matriciale e per singolo nodo. Da dove viene $\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$?

D1.L3 Che cos'è l'over-smoothing e da che cosa è causato a livello spettrale? Distinguiamo dall'over-squashing.

D2.L1 Forward e reverse process di un diffusion model.

D2.L2 Perché la reverse vera è intrattabile, e perché condizionandola su $\mathbf{x}_0$ diventa gaussiana?

D2.L3 Classifier guidance contro classifier-free: differenze, vantaggi, e il trade-off fedeltà/diversità.

Chapter 2

Risposte

Le risposte qui sotto sono *scheletri*: i passaggi obbligati, le formule che devono comparire, e il punto in cui si sbaglia. Per la trattazione estesa il rimando è a [Risposte_GDL.pdf](#), che contiene tutte e 235 le risposte per esteso.

Simulazione 1 — Self-attention

L1

Un token è un vettore: l'embedding di un elemento della sequenza. Non la parola, non l'indice nel vocabolario. La self-attention contestualizza ogni token in funzione di tutti gli altri della stessa sequenza. **L2**

$\mathbf{Q} = \mathbf{XW}^Q$, $\mathbf{K} = \mathbf{XW}^K$, $\mathbf{V} = \mathbf{XW}^V$, con $\mathbf{W}$ matrici *apprese*; poi $\text{Attention} = \text{softmax}(\mathbf{QK}^\top / \sqrt{d_k})\mathbf{V}$. Se *tutte e tre* vengono dalla stessa $\mathbf{X}$ è self-attention; se $\mathbf{Q}$ viene da un'altra sequenza è cross-attention. **L3**

Con componenti a varianza unitaria, $\mathbf{q} \cdot \mathbf{k}$ ha varianza d_k e deviazione standard $\sqrt{d_k}$. Senza scalatura la softmax *satura*: distribuzione quasi one-hot, derivata $\alpha_i(\delta_{ij} - \alpha_j)$ vicina a zero, gradiente che non passa. Con $d_k = 64$: punteggi (8, 0, 5.6) danno $\alpha = (0.917, 0.0003, 0.083)$, entropia 0.289 nat; divisi per 8 danno (0.474, 0.175, 0.351), entropia 1.026. (*Cap. 18 D1, D3.*)

Simulazione 2 — HMM + VAE

D1.L1

Parametri: $\pi_i = P(z_1=i)$, $A_{ij} = P(z_t=j \mid z_{t-1}=i)$, $B_{ik} = P(x_t=k \mid z_t=i)$. **D1.L2**

$P(x_{1:T}, z_{1:T}) = P(z_1)P(x_1|z_1) \prod_{t \geq 2} P(z_t|z_{t-1})P(x_t|z_t)$. Parte da s_1 *perché non ha genitori*. Parametrizzata con indicatrici: si scrive ogni fattore come $\prod_i \pi_i^{[z_1=i]}$ eccetera, così il logaritmo trasforma le produttorie in sommatorie. **D1.L3**

$\gamma_t(i) = \alpha_t(i)\beta_t(i)/P(x_{1:T})$; $\xi_t(i, j) = \alpha_t(i)A_{ij}B_{jx_{t+1}}\beta_{t+1}(j)/P(x_{1:T})$. M-step: $\hat{\pi}_i = \gamma_1(i)$, $\hat{A}_{ij} = \sum_t \xi_t(i, j) / \sum_t \gamma_t(i)$, $\hat{B}_{ik} = \sum_{t: x_t=k} \gamma_t(i) / \sum_t \gamma_t(i)$. **Lagrange** perché i parametri sono distribuzioni e devono sommare a uno. (*Cap. 9 D3, D4.*) **D2.L1**

$$-\mathcal{L} = -\mathbb{E}_q[\log p_\theta(\mathbf{x}|\mathbf{z})] + D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})). \quad \mathbf{D2.L2}$$

Jensen su $\log \int q \frac{p(\mathbf{x}, \mathbf{z})}{q}$; poi $p(\mathbf{x}, \mathbf{z}) = p(\mathbf{x}|\mathbf{z})p(\mathbf{z})$ separa i due termini. KL chiusa: $\frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$. **D2.L3**

Il campionamento è un nodo stocastico non differenziabile in ϕ . Si scrive $\mathbf{z} = \boldsymbol{\mu}_\phi + \boldsymbol{\sigma}_\phi \odot \boldsymbol{\epsilon}$: la casualità esce dal cammino del gradiente. Senza, resta lo score function estimator (REINFORCE), non distorto ma con varianza di ordini di grandezza superiore. (Cap. 20 D2, D3.)

Simulazione 3 — BN + esercizio + VAE

D1.L1

$P(\mathbf{x}) = \prod_i P(x_i | \text{pa}(x_i))$; costo $\sum_i (k-1)k^{|\text{pa}|}$ contro $k^N - 1$. **D1.L2**

Percorso $A \rightarrow B \leftarrow C \rightarrow D$: B è collider, C è fork. $A \perp D$ (collider chiuso). $A \perp D | C$ (fork chiuso, collider ancora chiuso). $A \not\perp D | B$ (il collider si apre). $A \perp D | \{B, C\}$ (il fork richiude). Con $B \rightarrow E$: $A \not\perp D | E$, perché E è discendente del collider e lo apre — è l'errore più frequente. **D1.L3**

No, non solo locali: la local Markov property ($X_i \perp \text{ND}(X_i) | \text{pa}(X_i)$) è equivalente alla fattorizzazione, ma la *d-separazione* è la versione globale ed è più forte, perché copre insiemi condizionanti arbitrari. (Cap. 3 D2, Cap. 4 D1–D2.) **D2**

Vedi Simulazione 2 per l'ELBO. Il termine KL forza i posterior a sovrapporsi e a coprire il prior: senza, il latente è un insieme di isole e campionare da $p(\mathbf{z})$ produce spazzatura. Posterior collapse: $q_\phi \rightarrow p(\mathbf{z})$ per ogni $\mathbf{x}$, KL a zero, decoder che ignora il latente; favorito da decoder troppo espressivi; si mitiga con KL annealing e free bits. (Cap. 19 D5, Cap. 20 D4.)

Simulazione 4 — CNN + HMM + Bellman

D1

$O = \lfloor (I - K + 2P)/S \rfloor + 1$. Parameter sharing: stesso kernel ovunque $\Rightarrow$ parametri indipendenti dalla dimensione dell'input ed **equivarianza** (non invarianza) alla traslazione; l'invarianza la dà il pooling. **D2**

$\alpha_1(i) = \pi_i B_{ix_1}$; $\alpha_t(i) = [\sum_j \alpha_{t-1}(j) A_{ji}] B_{ix_t}$; $P(x_{1:T}) = \sum_i \alpha_T(i)$. Complessità $\mathcal{O}(K^2 T)$ contro $\mathcal{O}(K^T)$ dell'enumerazione: tutti i cammini che passano per lo stesso stato allo stesso istante si sommano una volta sola. **D3**

$v^*(s) = \max_a \sum_{s'} P(s'|s, a) [R + \gamma v^*(s')]$. **Non lineare** perché max non è un operatore lineare (la expectation invece è un sistema lineare risolubile in forma chiusa). Contrazione: $\|\mathcal{T}u - \mathcal{T}v\|_\infty \leq \gamma \|u - v\|_\infty$, usando $|\max f - \max g| \leq \max |f - g|$ e il fatto che P somma a uno. Da Banach: punto fisso unico, convergenza da qualunque inizializzazione, geometrica di ragione γ . (Cap. 14, Cap. 9 D1, Cap. 29 D5–D6.)

Simulazione 5 — LSTM + Normalizing Flow

D1.L1

Due stati che viaggiano in parallelo: la **cella** $\mathbf{c}_t$ (la memoria, che scorre in orizzontale quasi indisturbata) e lo **stato nascosto** $\mathbf{h}_t$ (l'uscita). Tre porte, tutte sigmoidi che leggono $[\mathbf{h}_{t-1}, \mathbf{x}_t]$: *forget* (quanto della cella vecchia tenere), *input* (quanto del candidato nuovo scrivere), *output* (quanto della cella far uscire). Più il candidato $\tilde{\mathbf{c}}_t$, che è una tanh e non una

porta. Nel disegno: la cella è la linea dritta in alto, le porte sono i tre rubinetti che la toccano. **D1.L2**

$\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t = \sigma(\cdot)$, $\tilde{\mathbf{c}}_t = \tanh(\cdot)$, $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$, $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$. Additivo perché così la derivata lungo la cella è $\mathbf{f}_t$, un numero *scelto dalla rete*, invece di un prodotto di Jacobiani governato dal raggio spettrale. **D1.L3**

$\partial \mathbf{c}_t / \partial \mathbf{c}_{t-\tau} \approx \prod \text{diag}(\mathbf{f}_i)$. Con $f = 0.95$ su 50 passi: 0.081, contro 5.7×10^{-3} di una vanilla con $w = 0.9$. Con $\mathbf{f}_t \approx 0$ la memoria lunga sparisce: utile ai confini (fine frase, fine episodio, cambio di regime). **D2.L1**

Si parte da una densità semplice e nota p_Z (di solito $\mathcal{N}(\mathbf{0}, \mathbf{I})$) e le si applica una trasformazione f **invertibile e differenziabile**. Si fonda sulla formula del cambio di variabile: la densità trasformata si calcola *esattamente*, senza bound. È l'unica famiglia generativa profonda che ottimizza la log-verosimiglianza esatta invece di un surrogato (VAE: ELBO; GAN: nessuna likelihood; diffusion: ELBO). **D2.L2**

$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}) - \log |\det J_f(\mathbf{z})|$. Il determinante è il fattore di dilatazione locale del volume: se f espande, la stessa massa si spalma su più spazio e la densità *cala*, da cui il segno meno. Verifica: $z \sim \mathcal{N}(0, 1)$, $x = 2z + 1 \Rightarrow \log p_X(3) = -0.5 - 0.919 - 0.693 = -2.112 = \log \mathcal{N}(3|1, 4)$. **D2.L3**

Coupling (NICE/RealNVP): $\mathbf{x}_1 = \mathbf{z}_1$, $\mathbf{x}_2 = \mathbf{z}_2 \odot e^{s(\mathbf{z}_1)} + t(\mathbf{z}_1)$; Jacobiano triangolare perché il primo blocco non dipende dal secondo, quindi $\log |\det J| = \sum_j s_j$, in $\mathcal{O}(d)$, *qualunque* cosa siano s e t . Autoregressive (MAF/IAF), residual flow con $\text{Lip}(g) < 1$ e serie $\sum_k \frac{(-1)^{k+1}}{k} \text{tr}(J_g^k)$ stimata con Hutchinson, continuous (Neural ODE) con la traccia. (Cap. 16 D1–D3, Cap. 22 D1–D3, D8.)

Simulazione 6 — LDA + GNN

D1.L1

$\beta_k \sim \text{Dir}(\eta)$; per documento $\theta_d \sim \text{Dir}(\alpha)$; per posizione $z_{dn} \sim \text{Cat}(\theta_d)$, $w_{dn} \sim \text{Cat}(\beta_{z_{dn}})$. Solo le parole sono osservate. **D1.L2**

Il posterior $P(\theta, \mathbf{z} | \mathbf{w})$ è intrattabile; mean field $q(\theta, \mathbf{z}) = q(\theta | \gamma) \prod_n q(z_n | \phi_n)$, con $\phi_{dnk} \propto \exp(\mathbb{E}[\log \theta_{dk}] + \mathbb{E}[\log \beta_{k,w}])$ e $\gamma_{dk} = \alpha_k + \sum_n \phi_{dnk}$. Le digamma vengono da $\mathbb{E}_q[\log \theta_{dk}] = \Psi(\gamma_{dk}) - \Psi(\sum_j \gamma_{dj})$. **D1.L3**

Collapsed: si integrano θ e β per coniugazione, restano solo le z .

$$P(z_{dn} = k | \cdot) \propto (n_{d,k}^{-dn} + \alpha_k) \cdot \frac{n_{k,w}^{-dn} + \eta}{n_{k,\cdot}^{-dn} + V\eta}.$$

Prodotto di due pressioni: quanto il documento ama k , quanto k ama la parola. (Cap. 11, Cap. 12 D5.) **D2**

$\mathbf{m}_v = \text{AGG}(\{\mathbf{h}_u : u \in N(v)\})$, $\mathbf{h}'_v = \text{UPD}(\mathbf{h}_v, \mathbf{m}_v)$. AGG deve essere **invariante per permutazione** — altrimenti il modello dipenderebbe dalla numerazione arbitraria dei nodi — accettare cardinalità variabili ed essere differenziabile. Somma contro media: su $\{\{1, 1, 1\}\}$ e $\{\{1, 1\}\}$ media e max danno 1 in entrambi i casi, la somma 3 e 2: media e max perdono la molteplicità. Solo la somma (GIN, con MLP e ϵ) raggiunge il limite 1-WL. (Cap. 27 D1, D6.)

Simulazione 7 — Boltzmann + GAN

D1.L1

Una Boltzmann machine è un modello *a energia* su unità binarie, completamente connesso fra visibili e nascoste e al loro interno: $E(\mathbf{v}, \mathbf{h}) = -\mathbf{b}^\top \mathbf{v} - \mathbf{c}^\top \mathbf{h} - \mathbf{v}^\top \mathbf{W} \mathbf{h} - \mathbf{v}^\top \mathbf{L} \mathbf{v} - \mathbf{h}^\top \mathbf{J} \mathbf{h}$, con $P(\mathbf{v}, \mathbf{h}) = e^{-E}/Z$. **Restricted** significa che si tolgono le connessioni intra-strato ($\mathbf{L} = \mathbf{J} = \mathbf{0}$): resta un grafo *bipartito*, quindi $E(\mathbf{v}, \mathbf{h}) = -\mathbf{b}^\top \mathbf{v} - \mathbf{c}^\top \mathbf{h} - \mathbf{v}^\top \mathbf{W} \mathbf{h}$. È un MRF, non una rete Bayesiana: nessuna direzione, e la normalizzazione $Z = \sum_{\mathbf{v}, \mathbf{h}} e^{-E}$ è globale e intrattabile (2^{n+m} termini). **D1.L2**

$\partial \ell / \partial W_{ij} = \langle v_i h_j \rangle_{\text{data}} - \langle v_i h_j \rangle_{\text{model}}$. $\langle \cdot \rangle$ è il **valore atteso** rispetto alla distribuzione indicata a pedice — notazione dalla meccanica statistica. La fase negativa è difficile perché è un'attesa *sotto il modello*: richiede campioni dal modello, quindi Z . **D1.L3**

$P(h_j=1|\mathbf{v}) = \sigma(c_j + \sum_i v_i W_{ij})$, $P(v_i=1|\mathbf{h}) = \sigma(b_i + \sum_j W_{ij} h_j)$. La bipartizione rende le unità di uno strato condizionalmente indipendenti dato l'altro, quindi si campiona un intero strato in parallelo. **D2.L1**

$\min_G \max_D \mathbb{E}_{p_{\text{data}}} [\log D] + \mathbb{E}_{p_z} [\log(1 - D(G(z)))]$; $D^* = p_{\text{data}} / (p_{\text{data}} + p_G)$ massimizzando puntualmente $a \log d + b \log(1 - d)$. **D2.L2**

Si scrive $\sup$ e non $\max$ perché l'estremo superiore può non essere *raggiunto* da alcun elemento dell'insieme: il $\max$ esiste solo se il $\sup$ appartiene all'insieme. Su spazi di funzioni non compatti l'ottimo può non essere attinto. **D2.L3**

Kantorovich-Rubinstein: $W(p, q) = \sup_{\|f\|_L \leq 1} \mathbb{E}_p[f] - \mathbb{E}_q[f]$. È un *sup su una classe di funzioni* — non c'è garanzia che il massimo sia raggiunto, da cui il $\sup$. Il vincolo $\|f\|_L \leq 1$ è *costitutivo della dualità*: senza, il $\sup$ sarebbe $+\infty$ (basta scalare f). Quindi D deve essere Lipschitz non perché “sia” contrattivo, ma perché la formula che si sta usando *vale solo* sulle 1-Lipschitz. Realizzazioni: weight clipping, gradient penalty, spectral normalization. (Cap. 13 D3–D4, Cap. 21 D1, D5.)

Simulazione 8 — RNN + ELBO

D1

Weight sharing: stessi pesi a ogni istante $\Rightarrow$ numero di parametri indipendente dalla lunghezza, e possibilità di processare sequenze di lunghezza variabile. **L'assunzione è la stazionarietà**: ciò che vale al tempo t vale a qualunque altro t , quindi la conoscenza è trasferibile nel tempo. $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-\tau} = \prod \mathbf{W}^\top \text{diag}(1 - \tanh^2)$. Tanh e sigmoide fanno perdere informazione perché la loro derivata è ≤ 1 ($e \leq 1/4$ per la sigmoide), quindi *ogni* fattore della catena attenua. Il raggio spettrale mi informa **attraverso lo Jacobiano**: $\rho(\mathbf{W}) < 1$ vanishing, > 1 exploding, e la finestra utile è strettissima ($0.9^{49} = 5.7 \times 10^{-3}$, $1.1^{49} = 107$). **D2**

$\log p(x) = \mathcal{L}(q) + D_{\text{KL}}(q(z) \| p(z|x))$: poiché la KL è ≥ 0 , $\mathcal{L}$ è un lower bound; poiché $\log p(x)$ non dipende da q , massimizzare $\mathcal{L}$ è minimizzare la KL; e il gap è l'errore di approssimazione. EM: q = posterior esatto, bound tight. VI: q in una famiglia, bound non tight. VAE: q prodotta da una rete (ammortizzata). Diffusion: ELBO gerarchico su T latenti con encoder fisso. (Cap. 15 D1, D7, Cap. 10 D1–D2.)

Simulazione 9 — MRF + Normalizing Flow

D1

$P(\mathbf{x}) = \frac{1}{Z} \prod_c \psi_c(\mathbf{x}_c)$. Hammersley-Clifford: per $P > 0$ strettamente, Markov rispetto a $G \iff$ fattorizza sulle clique massimali. La positività non è tecnica: esistono controesempi con zeri (Moussouris). BN e MRF non si contengono: la v-structure $X \rightarrow Z \leftarrow Y$ non è rappresentabile da un MRF; il ciclo a 4 senza corde non è rappresentabile da un DAG. Conversione con moralizzazione, che perde proprio la v-structure. **D2**

$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}_0) - \sum_k \log |\det J_{f_k}|$: i log-determinanti si sommano. Requisiti: invertibilità con inversa calcolabile, determinante in tempo lineare, espressività. Perché non hanno vinto: invertibilità (niente ReLU non invertibili, niente pooling), **dimensione preservata** (nessun bottleneck, tutto alla piena dimensione dei dati), e struttura triangolare forzata che tocca metà componenti per layer. A parità di budget la qualità resta indietro. (Cap. 13 D1–D2, Cap. 22 D2, e la Trappola del Cap. 22.)

Simulazione 10 — MDP + Q-learning + LDA

D1

$(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. “Markov”: lo stato è *statistica sufficiente* del passato — assunzione sulla rappresentazione, non sul mondo (un frame di Atari non è markoviano, quattro impilati sì). γ : preferenza temporale e convergenza della serie più contrazione dell’operatore. Bellman: da $G_t = r_{t+1} + \gamma G_{t+1}$, prendi l’attesa e usa Markov per riconoscere $\mathbb{E}[G_{t+1}|s_{t+1}] = v_\pi(s')$. **D2**

$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$. Off-policy per il max: il target usa l’azione *greedy*, non quella che la politica di comportamento prenderà. SARSA usa a' effettivamente scelta. Numeri: $Q = 2.0$, $r = 1$, $\gamma = 0.9$, $\alpha = 0.1$, $\max Q' = 3.0$, $Q'_{\text{presa}} = 1.0 \Rightarrow$ Q-learning **2.17**, SARSA **1.99**. Cliff walking per raccontarlo. (Cap. 29–30.) **D3**

Vedi Simulazione 6.

Simulazione 11 — CNN + RBM

D1

Nelle reti si implementa la *cross-correlazione* (kernel non ribaltato): siccome il kernel è appreso, ribaltarlo non cambia l’insieme delle funzioni rappresentabili. Degradation problem: aggiungendo layer peggiora l’errore di *training* — non overfitting, ma ottimizzazione: la rete non riesce a imparare l’identità. ResNet impara il residuo, così l’identità corrisponde a $\mathcal{F} = 0$. Skip: $\partial \mathbf{y} / \partial \mathbf{x} = \mathbf{I} + \partial \mathcal{F} / \partial \mathbf{x}$, esiste sempre un cammino non attenuato. RF: $\text{RF}_\ell = \text{RF}_{\ell-1} + (k_\ell - 1)j_{\ell-1}$, $j_\ell = j_{\ell-1}s_\ell$; cinque conv 3×3 danno 11; con un pooling in mezzo il jump raddoppia e l’RF cresce molto più in fretta. **D2**

Vedi Simulazione 7 per energia e condizionali. CD- k : parte *dal dato*, k passi di Gibbs, usa lo stato risultante al posto di un campione dal modello. **CD-1 non è il gradiente di nessuna funzione obiettivo**: è distorto per due ragioni, k passi non bastano e partire dal dato tiene la catena in una regione favorevole. Persistent CD mantiene la catena fra mini-batch invece di ripartire dal dato. (Cap. 14, Cap. 13 D4.)

Simulazione 12 — LSTM + Diffusion

D1

GRU: fonde cella e stato, due porte, input e forget legati in $\mathbf{z}_t$, niente output gate, 3 matrici contro 4. A risorse limitate: GRU. **D2.L1**

$q(\mathbf{z}_t|\mathbf{z}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\mathbf{z}_{t-1}, \beta_t\mathbf{I})$; per induzione, sommando le varianze di due gaussiane indipendenti, $q(\mathbf{z}_t|\mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I})$. **D2.L2**

L'ELBO, i tre termini:

$$-\text{ELBO} = \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{z}_T|\mathbf{x}_0)||p(\mathbf{z}_T))}_{L_T \text{ costante}} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(\mathbf{z}_{t-1}|\mathbf{z}_t, \mathbf{x}_0)||p_\theta(\mathbf{z}_{t-1}|\mathbf{z}_t))}_{L_{t-1} \text{ denoising}} - \underbrace{\log p_\theta(\mathbf{x}_0|\mathbf{z}_1)}_{L_0 \text{ ricostruzione}} \right].$$

D2.L3

Si riscrive $\tilde{\mu}_t$ in funzione di ϵ , si parametrizza uguale, e $L_{t-1} \propto \|\epsilon - \epsilon_\theta\|^2$. Si trascurano i pesi λ_t , L_0 e L_T . **Non è neutro**: i pesi originali danno importanza ai t piccoli; pesare uniformemente sposta l'attenzione sui t grandi, migliora la qualità percettiva e *peggiora* la likelihood. (Cap. 16 D4, Cap. 23 D1, D3, D4.)

Simulazione 13 — GMM + CNN

D1

$P(x) = \sum_k \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$. E-step: $r_{nk} = \pi_k \mathcal{N}_k / \sum_j \pi_j \mathcal{N}_j$. M-step: $\mu_k = \sum_n r_{nk} x_n / N_k$, Σ_k analoga, $\pi_k = N_k / N$ (con Lagrange per il vincolo). *Sono le formule del caso completamente osservato con i conteggi interi sostituiti da conteggi frazionari*. Monotonia: $\log p(x|\theta^{\text{old}}) = \mathcal{L}(q^*, \theta^{\text{old}}) \leq \mathcal{L}(q^*, \theta^{\text{new}}) \leq \log p(x|\theta^{\text{new}})$; il passaggio che regge tutto è il **primo**, cioè che l'E-step azzerava la KL e rende il bound tight. **D2**

Input $32 \times 32 \times 3$, 6 filtri 5×5 , stride 1: output $28 \times 28 \times 6$; parametri $(5 \cdot 5 \cdot 3 + 1) \cdot 6 = \mathbf{456}$ contro $3072 \times 4704 = \mathbf{14\,450\,688}$ di una fully connected equivalente — fattore $\approx 31\,700$, e i 456 non dipendono dalla dimensione dell'immagine. (Cap. 8, Cap. 14 D1–D3.)

Simulazione 14 — Grafi + Diffusion

D1.L1

Una CNN presuppone ordine canonico dei vicini, grado costante e griglia regolare: un grafo non ha nessuna delle tre. Serve un'aggregazione invariante per permutazione che accetti insiemi di cardinalità variabile. **D1.L2**

$\mathbf{H}' = \sigma(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H} \mathbf{W})$; per nodo, $\mathbf{h}'_v = \sigma(\sum_{u \in N(v) \cup \{v\}} \frac{1}{\sqrt{\tilde{d}_v \tilde{d}_u}} \mathbf{W} \mathbf{h}_u)$. Viene dal Laplaciano normalizzato, approssimazione di Chebyshev al prim'ordine con $\theta_0 = -\theta_1$, più self-loop e *renormalization trick* (senza, gli autovalori in $[0, 2]$ rendono instabile l'iterazione). **D1.L3**

Over-smoothing: $\mathbf{S}$ ha autovalore dominante 1 con autovettore $\propto \sqrt{\tilde{\mathbf{d}}}$, quindi $\mathbf{S}^L$ proietta tutto lì e l'informazione delle feature sparisce (energia di Dirichlet da 1.736 a 4.2×10^{-6} in venti layer). È *omogeneizzazione*, causa spettrale. Over-squashing è *perdita di banda*, causa

topologica: un vicinato esponenziale compresso in un vettore fisso attraverso i colli di bottiglia del grafo. **D2.L1**

Forward (fisso, senza parametri): $q(\mathbf{z}_t | \mathbf{z}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{z}_{t-1}, \beta_t \mathbf{I})$, che in forma chiusa dà $\mathbf{z}_t = \sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}$; per T grande $q(\mathbf{z}_T) \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$. **Reverse** (appreso): $p_\theta(\mathbf{z}_{t-1} | \mathbf{z}_t) = \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{z}_t, t), \Sigma_t)$, percorso da $\mathbf{z}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ fino a $\mathbf{x}_0$. Il punto chiave: se i β_t sono *piccoli*, anche il reverse è gaussiano — è questo che rende il modello parametrizzabile. **D2.L2**

$q(\mathbf{z}_{t-1} | \mathbf{z}_t)$ richiederebbe la marginale $q(\mathbf{z}_{t-1})$, che dipende da p_{data} : il denoising è ambiguo. Ma $q(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{x}_0)$ è gaussiana in forma chiusa, con $\tilde{\boldsymbol{\mu}}_t$ e $\tilde{\beta}_t$ noti — e su quella si costruisce l'ELBO, perché in training $\mathbf{x}_0$ c'è. **D2.L3**

Classifier guidance: modello incondizionato più classificatore *su input rumorosi*, $\boldsymbol{\mu}' = \boldsymbol{\mu}_\theta + \gamma \sigma_t^2 \nabla \log p_\phi(\mathbf{y} | \mathbf{z}_t)$; richiede un modello in più, gradienti instabili, mal si presta al testo. Classifier-free: un solo modello con dropout della condizione, $\hat{\boldsymbol{\epsilon}} = (1 + \gamma) \boldsymbol{\epsilon}_y - \gamma \boldsymbol{\epsilon}_\emptyset$, equivalente a $(1 - w) \boldsymbol{\epsilon}_\emptyset + w \boldsymbol{\epsilon}_y$ con $w = 1 + \gamma$. Trade-off: più guidance, più fedeltà alla condizione, *meno* diversità. (Cap. 26–28, Cap. 23 D2, D5.)